

SMAI Assignment 2

Q1. Bike Rental Demand Prediction with Linear Regression [50 marks]

Predicting short-term rental demand is a real operational problem for bike-share systems. It drives rebalancing bikes across stations and staffing decisions. In this question you will predict the hourly count of bike rentals from time and weather features using linear regression.

Dataset: UCI Bike Sharing Dataset. Use `hour.csv`, available at <https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset>

Relevant columns: `season`, `yr`, `mnth`, `hr`, `holiday`, `weekday`, `workingday`, `weathersit`, `temp`, `atemp`, `hum`, `windspeed`, and `cnt`, the target, which is the total number of rentals in that hour. The file also contains `casual` and `registered`, which sum exactly to `cnt`; you must not use them as features.

You may not use any library implementation of linear regression. Split the dataset into training, validation and test sets using a random 80:10:10 split. This is hourly time-series data. Consider what a random split implies about your validation set, and state this in one or two sentences alongside your split.

1.1 Feature Engineering [9 marks]

Several raw features, including hour of day, month, and day of week, are recorded as plain integers. Consider whether that representation is appropriate for a linear model, and design features that address any issues you find, justifying each one. You may consider: one-hot encodings of categorical columns like `season` and `weathersit`; interaction terms between weather and time of day (limit yourself to at most five such interactions, and justify your choices).

To check that your representation of hour is actually useful, fit two versions of your model that are identical except for how hour is represented: one using `hr` as a single raw integer, and one using your chosen encoding. Report the validation RMSE for each, and state what the gap tells you about what the raw-integer encoding forces the model to assume.

Report the number of columns in your final design matrix.

1.2 Dataset Preprocessing and Analysis [14 marks]

1.2.1 Train/val/test split [2 marks]

Report the number of rows in your training, validation, and test sets.

1.2.2 Feature normalization [8 marks]

Standardize continuous features. Then, using the gradient descent implementation from 1.3.2, run it with and without standardization at a few learning rates, and report the largest learning rate that remains stable in each case, and the number of iterations each needs to converge at a learning rate that is stable for both. Explain what you observe.

Plot loss versus iteration for learning rates `{1e-5, 1e-4, 1e-3, 1e-2, 0.1, 1.0}`, using your standardized features. Describe what happens at each extreme.

1.2.3 Visualization [4 marks]

Produce at least four plots that help you understand the data before modeling it. Include the distribution of the target variable, the relationship between at least two individual features and the target, and a correlation heatmap across your engineered features. For each plot, state in one line what it tells you.

1.3 Linear Regression from Scratch [12 marks]

1.3.1 Closed-form solution [8 marks]

Implement the normal equation directly. Report the condition number of your design matrix X and of $X^T X$ before solving. Then compute the closed-form solution and report its training RMSE. If the condition number is very large, diagnose the source of the ill-conditioning. If it is caused by a redundant feature representation, correct the representation and report both condition numbers again.

1.3.2 Gradient descent [4 marks]

Implement batch gradient descent from scratch. Compare the converged weights against the closed-form solution as a correctness check.

1.4 Polynomial Features and the Bias-Variance Tradeoff [6 marks]

Bike-share demand is known to have a nonlinear relationship with temperature. The main dataset's temperature range does not isolate this relationship cleanly, so this part uses a companion dataset built to study it directly.

Dataset: `bike_temp_comfort_zone.csv`. Columns are `temp`, `demand`, and `split` (values `train`, `val`, `test`); use this column instead of performing your own split for this part.

Construct polynomial features of `temp` at degrees 1 through 12. Standardize them before fitting, that is, subtract the mean and divide by the standard deviation of each power, computed on the training set. Fit your closed-form linear regression at each degree. Plot training and validation error against degree on the same axes. Report the region of degrees where validation error is minimized, and describe what happens to both curves on either side of it.

Also plot the magnitude of the largest fitted coefficient against degree, excluding the intercept, on a logarithmic scale, over the same degree range. Report the test RMSE at the degree within that region with the lowest validation error.

1.5 Regularization [6 marks]

Implement Ridge (L2) regression from scratch.

Sweep the regularization strength across several orders of magnitude (for example `0`, `1e-2`, `1`, `10`, `100`, `1e3`, `1e4`, `1e5`, `1e6`) using your full training set, and plot training and validation error against it. Report how much, if at all, your best value improves validation error compared to no regularization.

Then repeat the same sweep using a random subsample of 100 rows from your training set. With only 100 rows, some engineered columns may be constant in the subsample; drop zero-variance columns before standardizing, and use a least-squares solver rather than an explicit matrix inverse for the `lambda = 0` fit. Again report how much, if at all, your best value improves validation error compared to

no regularization, and compare the two improvements. Explain why the two improvements differ.

1.6 Evaluation [3 marks]

Evaluate on the test set the ridge regression model from 1.5, using the regularization strength you selected from your full training set sweep. Report MAE, RMSE, R^2 , and Pearson correlation in a table. Note that a linear model may produce negative predictions for a non-negative count target; report your metrics without clipping, and comment briefly on this.

Submission: Provide a `report.md` file describing your approach, all results, and your answers to every question above, alongside your code.

Q2. Radar Signal Classification with Support Vector Machines [50 marks]

Support Vector Machines were, for a long time, the default choice whenever a classification problem needed a decision boundary that was more than a straight line but a full neural network was overkill. In this question, you will build one from scratch, in both of its standard forms, and use it to classify radar returns.

Dataset: UCI Ionosphere. Available at <https://archive.ics.uci.edu/dataset/52/ionosphere>

- 351 rows, 34 continuous columns (`a01` through `a34`), derived from 17 radar “pulse numbers” with 2 values recorded per pulse
- Target: whether the return showed evidence of structure in the ionosphere (“good”) or passed through with no detectable structure (“bad”)
- Treat “good” as the positive class throughout

Rules:

- You may not use any library implementation of a Support Vector Machine (this includes `sklearn.svm` and equivalents)
- You may use a general-purpose quadratic programming or convex optimization solver (for example `cvxopt`) to solve the dual optimization problem

in 2.2.2. But you must derive the optimization problem yourself, construct its inputs yourself, and extract the resulting model (support vectors, weights, bias) yourself

(If `pip install cvxopt` fails while building from source, use `pip install --only-binary :all: cvxopt`)

- You may not use any library implementation of an evaluation metric; implement confusion matrix, accuracy, precision, recall and F1 from scratch
- Split the dataset into a training set and a test set using an 80:20 ratio, stratified by label. Do not touch the test set until 2.5
- Select every hyperparameter in this question (learning rate, kernel, degree, gamma, C) using 5-fold stratified cross-validation on the training set. Report cross-validation accuracy as the mean across the five folds
- Fix your random seed and report it
- Wherever you need to decide which alphas count as support vectors, use the threshold `alpha_i > 1e-5` , and state the threshold you used

2.1 Preprocessing and Data Analysis [8 marks]

2.1.1 Inspecting the features [2 marks]

- Look at the summary statistics of each of the 34 columns before doing anything else
- If any of the columns carry no information, remove them and justify your decision in one or two lines
- State what would go wrong later in the pipeline if you left them in
- Report the number of feature columns you are left with

2.1.2 Train/test split and cross-validation folds [2 marks]

- Report the number of rows in your training and test sets
- Report the number of positive and negative examples in each
- Report how you constructed your 5 cross-validation folds, and the class balance within them

- State your random seed

2.1.3 Standardization [2 marks]

- Standardize your feature columns. Fit the mean and standard deviation on training data only, and apply them to the test set and to each cross-validation fold without refitting
- In one or two lines, explain why this matters not just for the linear model but for the kernels you will implement in 2.3

2.1.4 Visualization [2 marks]

- Produce at least three plots: the class balance, the relationship between at least one pair of features colored by class label, and a correlation heatmap across your retained features
- For each plot, state in one line what it tells you

2.2 Linear SVM from Scratch [15 marks]

2.2.1 Primal formulation: subgradient descent [5 marks]

- Implement the soft-margin linear SVM using batch or mini-batch subgradient descent. Sum the per-point subgradients over your batch rather than averaging them; the learning rates below assume this convention
- Try three fixed learning rates (`1e-4` , `1e-3` , `1e-2`) and one schedule that decreases over iterations
- Report what you observe with each, and justify the choice you settle on
- Report the values of any hyperparameters you used (C, learning rate/schedule, number of iterations)

2.2.2 Dual formulation: quadratic program [7 marks]

- Derive the dual of the soft-margin SVM problem, and cast it into the standard form expected by a generic QP solver (such as `cvxopt.solvers.qp`) — this means identifying `x` , `P` , `q` , `G` , `h` , `A` and `b` in terms of your training data, and turning the dual's maximization into the solver's minimization

- State what each of x , P , q , G , h , A and b corresponds to in your derivation
- Solve it, and from the solution extract the set of support vectors and the bias term
- Report the number of support vectors as a fraction of the training set, at the C value you settled on in 2.2.1
- For the linear kernel only, recover an explicit weight vector from the dual solution
- Compare it against the weight vector you obtained by subgradient descent in 2.2.1: report the largest elementwise difference between the two at three different iteration counts, for both your fixed learning rate and your decreasing schedule, and comment on the trend in each

2.2.3 Convergence and comparison [3 marks]

- Plot the objective versus iteration for the three learning rates considered in 2.2.1 and for your chosen decreasing schedule, on one set of axes over the same iteration range. Briefly discuss convergence behavior
- Report the training time of subgradient descent and the QP solver, and state which approach you expect to scale better with the number of training examples

2.3 Kernels [14 marks]

This section asks you to implement two kernel functions in place of the dot product in your dual formulation, and study what each one does to the decision boundary.

2.3.1 A case where linear fails [4 marks]

The Ionosphere features are 34-dimensional, which makes it hard to look at a decision boundary directly. This subsection asks you to find a two-feature view of the data where you can.

- Pick two of the Ionosphere columns (for example, just a_{03} and a_{08}) and refit your dual/QP solver from 2.2.2, with a linear kernel, using only those two as

features, ignoring the rest. Try several such pairs until you find one where the linear SVM fails to beat the accuracy of always predicting the majority class

- Report the pair you find and its linear SVM cross-validation accuracy
- Plot the data in that two-feature space, colored by class, together with your linear SVM's decision boundary
- Explain, in terms of the shape of the classes in that plot, why a linear boundary cannot do much better than this
- Confirm that at least one of the kernels you implement in 2.3.2 and 2.3.3 does substantially better on this same pair, and report its accuracy

2.3.2 Polynomial kernel [4 marks]

- Implement the polynomial kernel, and substitute it for the dot product wherever your dual formulation and QP setup from 2.2.2 use one
- Use $\gamma = 1 / (\text{number of retained features})$ and a constant offset term $\text{coef0} = 1$ throughout, and state the values you used
- Sweep the degree from 1 to 5
- On the feature pair you chose in 2.3.1, plot the decision boundary at each degree
- On the Ionosphere training and cross-validation folds (using all retained features), plot accuracy against degree
- Report the degree that minimizes cross-validation error, and describe what the boundary looks like on either side of it on your 2.3.1 feature pair

2.3.3 RBF kernel [4 marks]

- Implement the RBF kernel, and substitute it in the same way
- Sweep gamma across several orders of magnitude (for example $1e-3, 1e-2, 1e-1, 1, 10, 100$)
- Produce the same two kinds of plots as in 2.3.2 (decision boundary on the feature pair from 2.3.1, accuracy against gamma on Ionosphere using all retained features)

- Describe what the decision boundary looks like at the extremes of your gamma range, and what happens to the number of support vectors at each extreme

2.3.4 Comparative study [2 marks]

- Report a single table using cross-validation on the training set: linear kernel, your best polynomial degree, and your best gamma for RBF, each with cross-validation accuracy, number of support vectors, and training time for a single fit on the full training set
- State which kernel you would choose, and propose one explanation, grounded in the structure of the lonosphere features, for why it wins

2.4 The Regularization Parameter C [5 marks]

- Using the kernel you chose in 2.3.4, sweep C over `{1e-2, 1e-1, 1, 10, 100}` on your full training set
- Plot training and cross-validation error, and the number of support vectors, against C
- Report the value of C that minimizes cross-validation error, and how much it improves cross-validation error compared to the smallest C in your sweep. If several values of C tie, report the smallest of them
- Explain the shape of the curves: what C controls, and what happens to the number of support vectors as C grows

2.5 Evaluation [4 marks]

- Evaluate four configurations on the test set: your best linear SVM, your best polynomial SVM, your best RBF SVM, and a trivial classifier that always predicts the majority class
- Tune C by cross-validation separately for each of the three SVMs before this final evaluation, and report the C you used for each
- Report accuracy, precision, recall and F1 for the positive class for all four in a single table

- Given the class balance you reported in 2.1.2, comment on what the trivial baseline's accuracy conceals

2.6 Support Vectors and Error Analysis [4 marks]

- For your best overall model, report what fraction of the training set became support vectors, and state what that fraction suggests about the fit
- Report how many training points carry non-zero hinge loss under your best overall model and under your linear SVM from 2.2.2, and comment on the difference. Count a point as having non-zero hinge loss if its hinge loss exceeds `1e-3`
- Taking your linear SVM from 2.2.2, identify the training points with the largest hinge loss and inspect a handful of them
- State whether you find a pattern in which points end up here
- If you find one, propose one concrete change to your pipeline that you would expect to help, and state what you would expect to happen if you made it — you do not need to implement it
- If you find no pattern, say so and justify that from the points themselves

Total: 100 marks (Q1: 50, Q2: 50)